Programming Fundamentals Using C++

Huma Israr

Department Of Information Engineering Technology

National Skills University, Islamabad.

huma.israr@nsu.edu.pk

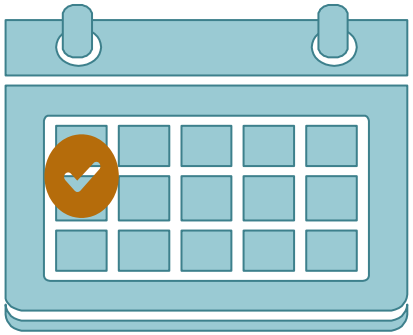
Fall 2024, BS-CS

Week 06

Lecture – 01

Lecture – 02

Control Structures (Loops)



Semester Calendar

Class Schedule

Lab Hours (Practice)

Tuesday (09:00 to 10:30)
Wednesday (09:30 to 12:30)
Wednesday (03:00 to 04:30)

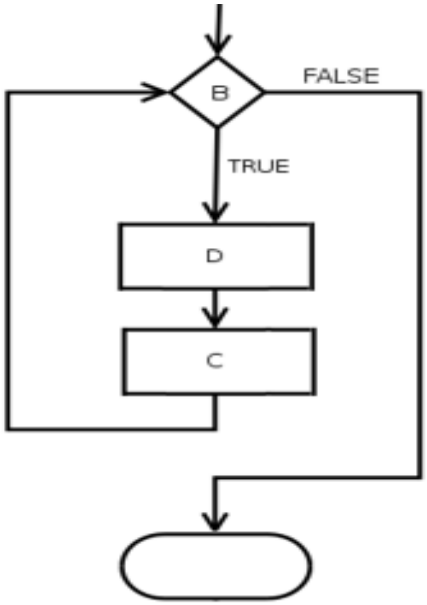
Control Structures

- Control Structures are just a way to specify flow of control in programs. It basically analyzes and chooses in which direction a program flows based on certain parameters or conditions. There are three basic types of logic, or flow of control, known as:

1. Sequence logic, or sequential flow
2. Selection logic, or conditional flow
3. Iteration logic, or repetitive flow

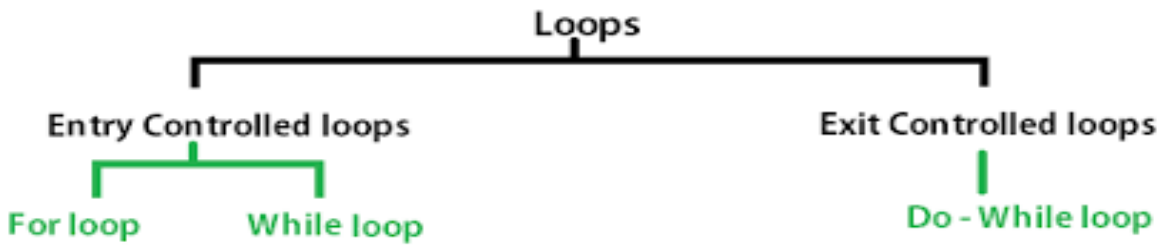
Iteration logic, or repetitive flow (Loop)

- In general, statements are executed sequentially: The first statement in a function is executed first, followed by the second, and so on. There may be a situation, when you need to execute a block of code several number of times.
- A loop statement allows us to execute a statement or group of statements multiple times.



Loop

C++ programming language provides the following type of loops to handle looping requirements.



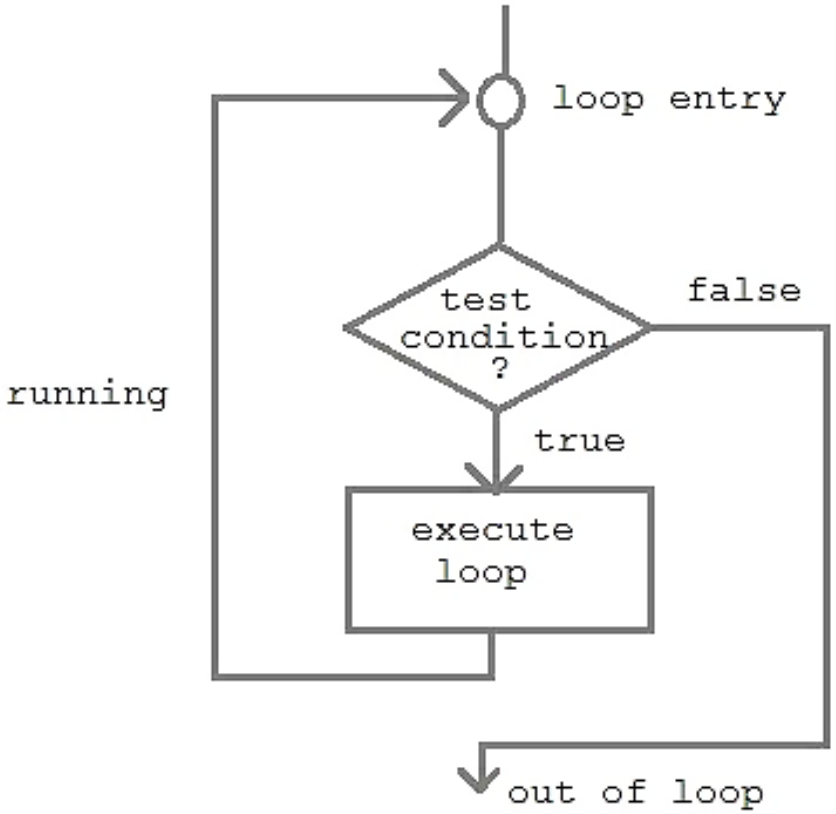
S.No.	Loop Type	Description
1	For Loop	Execute a sequence of statements multiple times and abbreviates the code that manages the loop variable.
2	While Loop	Repeats a statement or group of statements while a given condition is true. It tests the condition before executing the loop body.
3	Do-While loop	Like a 'while' statement, except that it tests the condition at the end of the loop body.
4	Nested Loop	using one or more loop inside any another 'while', 'for' or 'do.. while' loop.

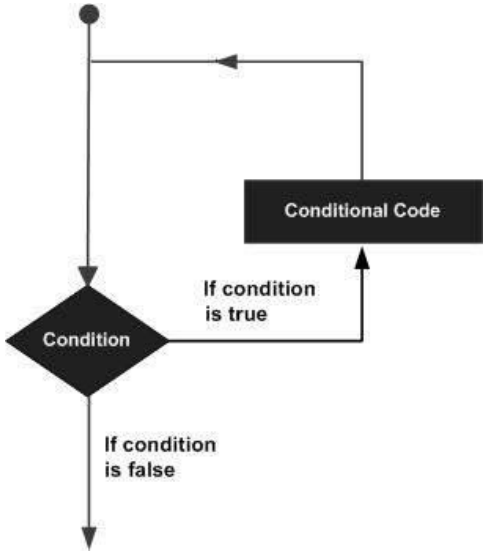
Loop Control Statements

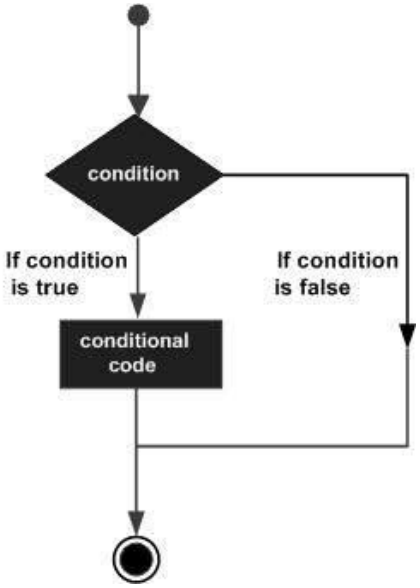
- Loop control statements change execution from its normal sequence. C++ language allows jumping from one statement to another within a loop as well as jumping out of the loop.

S.No.	C0ntrol Statement	Description
1	break	Terminates the loop or switch statement and transfers execution to the statement immediately following the loop or switch.
2	Continue	Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.
3	go-to	Transfers control to the labeled statement. Though it is not advised to use goto statement in your program.

How it Works:


loop entry
test condition ?
false
running
true
execute loop
out of loop


Conditional Code
If condition is true
Condition
If condition is false


condition
If condition is true
If condition is false
conditional code

For Loop

- When you know exactly how many times you want to loop through a block of code, use the for loop.

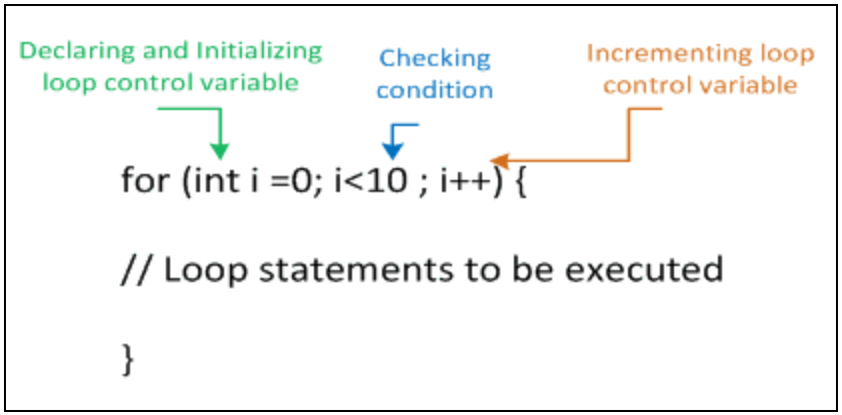
```
for (initialization; condition; update) {
   // body of-loop
}
```

1. Initialization: is executed (one time) before the execution of the code block.
2. Condition: defines the condition for executing the code block. if true, the body of for loop is executed if false, the for loop is terminated
3. Update: is executed (every time) after the code block has been executed. updates the value of initialized variables and again checks the condition.

For Loop:

Example

```
for (int i = 0; i < 10; i++) {
  cout << i << "\n";
}
```



- Explanation
- Statement 1: sets a variable before the loop starts (int i = 0).
- Statement 2: defines the condition for the loop to run (i must be less than 5). If the condition is true, the loop will start over again, if it is false, the loop will end.
- Statement 3 : increases a value (i++) each time the code block in the loop has been executed.

For Loop

```
// C++ Program for Printing
Numbers From 1 to 5
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 5; ++i) {
    cout << i << " ";
  }
  return 0;
}
```

```
// C++ Program to display a text 5
times
#include <iostream>
using namespace std;

int main() {
  for (int i = 1; i <= 5; ++i) {
    cout <<  "Hello World! " << endl;
  }
  return 0;
}
```

Comparison

```
#include <iostream>
using namespace std;
int main(){
    cout << "Hello World\n";
    cout << "Hello World\n";
    cout << "Hello World\n";
    cout << "Hello World\n";
    cout << "Hello World\n";
    cout << "Hello World\n";
    cout << "Hello World\n";
    cout << "Hello World\n";
    cout << "Hello World\n";
    cout << "Hello World\n";
    return 0;   }
```

```
#include <iostream>
using namespace std;

int main()
{
    for (int i = 1; i <= 10; i++)
    {
        cout << "Hello World\n";
    }

    return 0;
}
```

// C++ program to illustrate If statement

```
#include <iostream>
using namespace std;
int main()
{
    int i = 10;
    if (i < 15)
    {
        cout << "10 is less than 15 ";
    }
    cout << "I am Not in if";
    return 0;
}
```

// C++ program to illustrate If statement

```
#include <iostream>
using namespace std;
int main()
{
    int i = 10;
    if (i > 15)
    {
        cout << "10 is greater than 15 “;
    }
    cout << "I am Not in if";
    return 0;
}
```

While Loop:

- A loop is mainly consisted of three statements – initialization expression, test expression, update expression. The syntax of the three loops – For, while and do while mainly differs on the placement of these three statements.

```
initialization expression;
while (test_expression)
{
  // statements-1
 // statements-2
  update_expression;
}
```

```
// C++ program to illustrate while loop
#include <iostream>
using namespace std;
int main(){
   int i = 1;          // initialization expression
   while (i < 6)     // test expression
   {
      cout << "Hello World\n";
       i++;           // update expression
}
   return 0;
}
```

do while loop:

- In some situations it is necessary to execute body of the loop before testing the condition. Such situations can be handled with the help of do-while loop. do statement evaluates the body of the loop first and at the end, the condition is checked using while statement. General format of do-while loop is,

```
initialization expression;
do
{
  // statements
  update_expression;
} while (test_expression);
```

// C++ program to illustrate do-while loop

```
#include <iostream>
using namespace std;
int main()
{
    int i = 2;                  // Initialization expression
    do
    {
        // loop body
        cout << "Hello World\n";
        i++;                    // update expression
    }  while (i < 1);       // test expression
    return 0;
}
```

```
/ /C++ program to demonstrate infinite loops
// using for
#include <iostream>
using namespace std;
int main ()
{
  int i;
 for ( ; ; ) // This is an infinite for loop as the condition....expression is blank
    {
        cout << "This loop will run forever.\n";
    }
    // This is an infinite for loop as the condition
return 0;
}
```

```
#include <iostream>
using namespace std;
int main(){
  for(int i=1; i>=1; i++){
    cout<<"Value of variable i is: "<<i<<endl;
  }
  return 0;
}
```

```
/ /C++ program to demonstrate infinite loops
// using while…….
#include <iostream>
using namespace std;
int main (){
  int i;   // This is an infinite for loop as the condition
  //given in while loop will keep repeating infinitely
    while (i != 0)
    {
        i-- ;
        cout << "This loop will run forever.\n";
    }
    return 0;
}
```

```
/ /C++ program to demonstrate infinite loops
// using  while…….
#include <iostream>
using namespace std;
int main (){
  int i;
    // This is an infinite for loop as the condition
    // given in while loop is "true"
    while (true)
    {
        cout << "This loop will run forever.\n";
    }
    return 0;
}
```

```
#include <iostream>
using namespace std;
int main(){
  for(int i=1; i>=1; i++){
    cout<<"Value of variable i is: "<<i<<endl;
  }
  return 0;
}
```

Nested for loop:

- We can also have nested for loop, i.e one for loop inside another for loop. Basic syntax is,

```
for(initialization; condition; increment/decrement)
{
   for(initialization; condition; increment/decrement)
   {
      statement;
   }
}
```

Jumping out of a loop:

- Sometimes, while executing a loop, it becomes necessary to skip a part of the loop or to leave the loop as soon as certain condition becomes true, that is jump out of loop.

- break statement

When break statement is encountered inside a loop, the loop is immediately exited and the program continues with the statement immediately following the loop.

- continue statement

It causes the control to go directly to the test-condition and then continue the loop process. On encountering continue, cursor leave the current cycle of loop, and starts with the next cycle.

How break works

```
for (init; condition; update) {
    // code
    if (condition to break) {
        break;
    }
    // code
}
```

```
while (condition) {
    // code
    if (condition to break) {
        break;
    }
    // code
}
```

How Continue Works:

```
for (init; condition; update) {
    // code
    if (condition to break) {
        continue;
    }
    // code
}
```

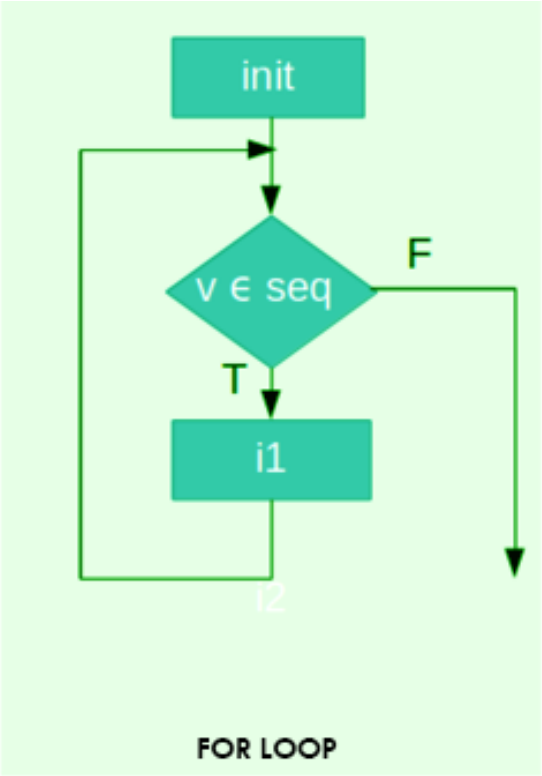
```
while (condition) {
    // code
    if (condition to break) {
        continue;
    }
    // code
}
```

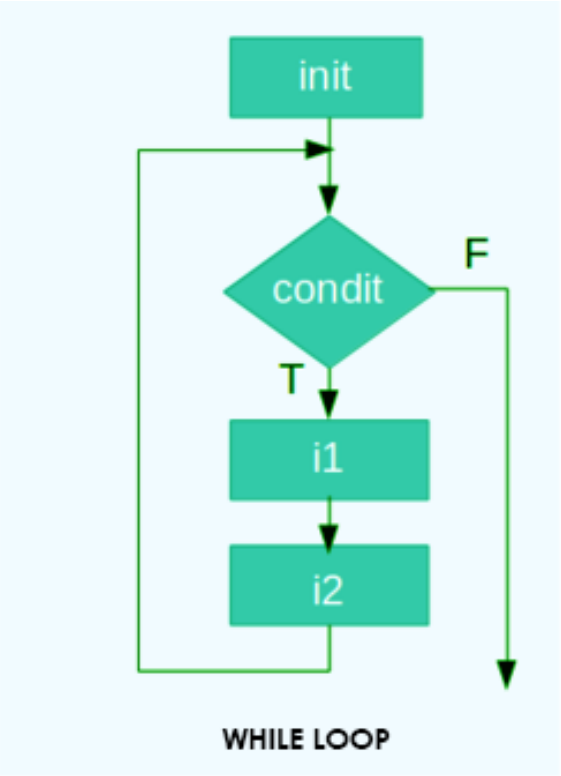
do while loop:

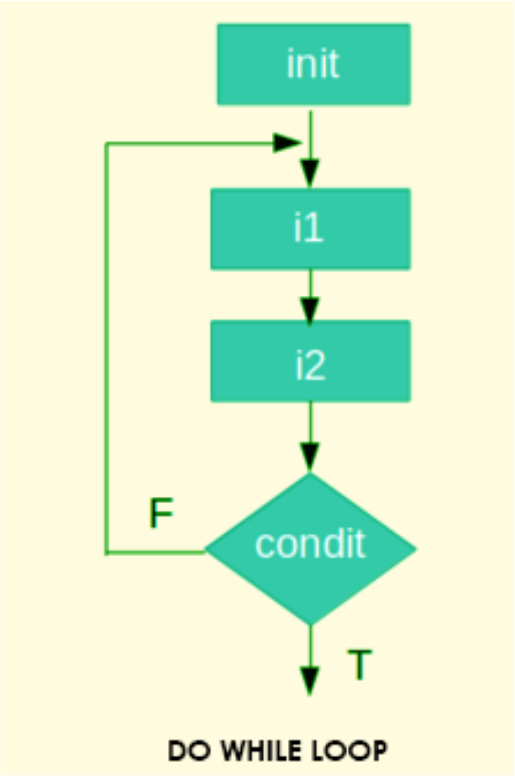
- In do while loops also the loop execution is terminated on the basis of test condition. The main difference between do while loop and while loop is in do while loop the condition is tested at the end of loop body, i.e do while loop is exit controlled whereas the other two loops are entry controlled loops.

```
initialization expression;
do
{
  // statements
  update_expression;
} while (test_expression);
```

Comparison:


init
v ∈ seq
F
T
i1
i2
FOR LOOP


init
condit
F
T
i1
i2
WHILE LOOP


init
i1
i2
F
condit
T
DO WHILE LOOP

Any Question